

N-Step SARSA —

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [G_{t:t+n} - Q(s_t, a_t)]$$

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^n Q(s_{t+n}, a_{t+n})$$

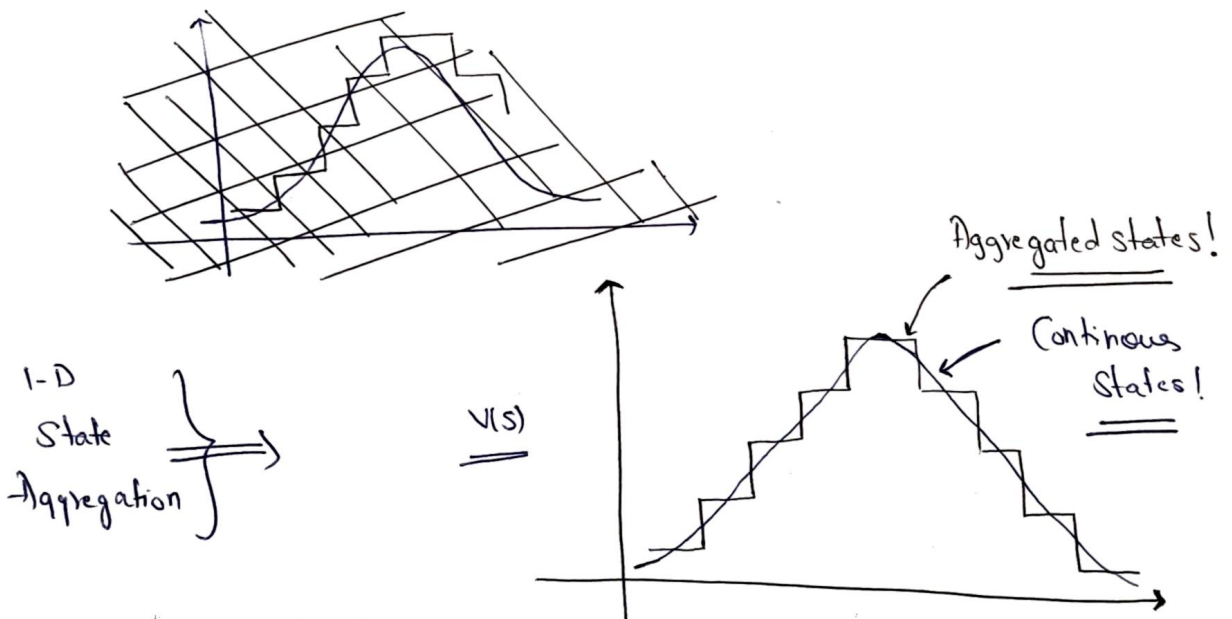
⇒ Continuous State Spaces

Convert the Continuous state space into a discrete state space!

$$\mathbb{R} \rightarrow \{s_0, s_1, \dots, s_n\}$$

1) State Aggregation 2) Tile Coding

1) State Aggregation!

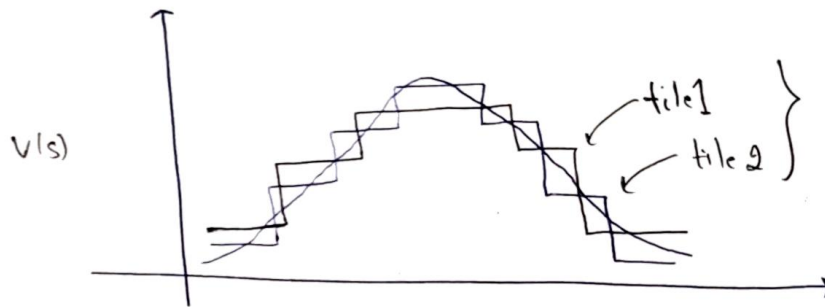


(9)

from going from Continuous states to aggregated states there is a precision loss—

2) Tile Coding—

Generalization of state aggregation in which 'n' independent aggregations are created!



Disadvantages —

1) limited Precision

2) Complexity ($O(n^k)$)

$n \equiv \# \text{ of Segments}$

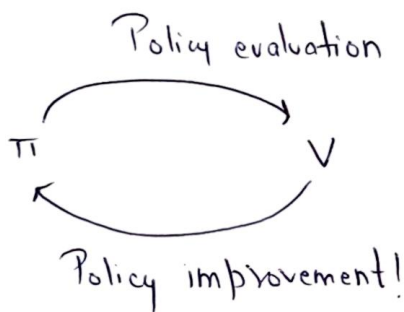
$k \equiv \# \text{ of Dimensions!}$

Functions Approximators :-

The functions $v_*(s)$ and $q_*(s,a)$ are not known in advance!

The agent learns based on experience!

$f_1(s|w) \rightarrow f_2(s|w) \rightarrow \dots \rightarrow f_n(s|w) \approx v_*(s)$



The policy evaluation cycle will modify the parameters on that function and obtain the new function!

State:

$$s = [s_1, s_2, \dots, s_n]$$

Parameters Vectors

$$w = [w_1, w_2, \dots, w_n]$$

1) linear approximators —

$$f(s|w) = \hat{v}(s|w) = w \cdot s^T$$

$$f(s) = \hat{v}(s) = w_1 s_1 + w_2 s_2 + \dots + w_n s_n$$

Inputs :

$$\phi(s) = [s_1, s_1^2, \dots, s_1^k, s_2^i, \dots, s_3^i, \dots, s_n]$$

Parameter vector:

$$w = [w_1, w_2, w_3, \dots, w_n]$$

2) Polynomial approximator —

$$f(s|w) = \hat{v}(s|w) = w \cdot \phi(s)^T$$

$$f(s) = \hat{v}(s) = \underline{w_1 s_1 + w_2 s_1^2 + \dots}$$

Policy Gradient Methods —

These methods use a function approximator to estimate the probabilities of taking each action

$$\pi(a|s, \theta) \in [0, 1]$$

Thus, in these methods the neural network is the policy! and these are stochastic!

Value based methods cannot represent stochastic policies in a simple way

1) Greedy policy!

$$\pi(a'|s) = \begin{cases} 1, & \text{if } a' = \arg\max_a \hat{q}(s, a|\theta) \\ 0, & \text{else} \end{cases}$$

2) Epsilon-greedy policy!

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \epsilon/|A|, & \text{if } a = \operatorname{argmax}_a \hat{q}(s,a|\theta) \\ \epsilon/|A|, & \text{else} \end{cases}$$

3) However, we can't represent
 $\pi^*(s) = [0.7, 0.3]$!

Advantage — The policy changes more smoothly
~~than~~ during learning —

Value based methods :-

When the max q value changes, they choose a new action 100% of the time

$$\boxed{a = \operatorname{argmax}_a \hat{q}(s,a)}$$

Policy Gradient Methods :-

The probability of choosing an action changes in small increments!

$$\boxed{a \sim \pi(s|\theta)}$$

Policy performance :-

Our goal is to find the θ values that maximize policy performance —

$$\boxed{\pi_*(a|s,\theta) = \operatorname{argmax}_{\theta} J(\theta)}$$

(11)

We will use the experience samples that the agent obtains to approximate the policy's performance as $\hat{J}(\theta)$!

We will approximate the optimal θ values by stochastic gradient ascent (SGA):

$$\underline{\theta_{t+1} = \theta_t + \alpha \nabla \hat{J}(\theta)}$$

$$\text{where, } \nabla \hat{J}(\theta) = \left[\frac{\partial \hat{J}(\theta)}{\partial \theta_1}, \frac{\partial \hat{J}(\theta)}{\partial \theta_2}, \dots, \frac{\partial \hat{J}(\theta)}{\partial \theta_n} \right]$$

The Policy Gradient theorem

define the policy performance as -

$$\underline{J(\theta) = v_{\pi}(s_0)}$$

$\nabla J(\theta) \propto \sum_s \underbrace{\mu(s)}_{\substack{\uparrow \\ \text{State distribution} \\ \text{following policy } \pi}} \sum_a \underbrace{q_{\pi}(s,a)}_{\substack{\uparrow \\ \text{q-value of a state action} \\ \text{pair following} \\ \text{policy } \pi.}} \underbrace{\nabla \pi(a|s,\theta)}_{\substack{\checkmark \\ \text{Gradient of the} \\ \text{probability selection} \\ \text{action } a.}}$

Gradient of policy performance.

the above expression means ... that the gradient of the performance of the policy is proportional to the return of each action in each state, multiplied by the gradient of the probability of taking that action in that state and weighted by the frequency with which we observe each state following that policy!

Intuitively, if an action in a state produces a low return to increase the return, we must increase the probability of taking that action!

REINFORCE :

(Policy Gradient + Monte Carlo)

Init, the agent will face a task for a complete episode to collect experience, and at the end of the episode it will update the neural network using all the experienced gathered

we will perform Stochastic gradient Ascent (SGA)

$$\theta_{t+1} = \theta_t + \alpha \nabla \hat{J}(\theta)$$

$\Rightarrow$ To do this we will have to approximate the gradient of the policy performance estimate

$\nabla \hat{J}(\theta)$ using samples collected from the environment!

Gradient of the performance estimate -

$$\left\{ \nabla \hat{J}(\theta) = r^t G_t \frac{\nabla \pi(A_t | s_t, \theta_t)}{\pi(A_t | s_t, \theta_t)} \right\}$$

please note ~~the~~ here, there is no bootstrapping. Instead, all the rewards obtained until the end of the episode will be used to compute the return, as MC methods do.

Multiplying the expression by ' r^t ' gives more weight to the updates corresponding to the initial actions and less weights to the final ones.

That's important b/c the initial actions have large effect on the outcome of the task!

$$\left\{ \theta_{t+1} = \theta_t + \alpha r^t G_t \frac{\nabla \pi(A_t | s_t, \theta_t)}{\pi(A_t | s_t, \theta_t)} \right\}$$

- # Also, the probability of taking that action increases inversely proportional to the probability that the neural network has to choose that action now —
- # That means that if the neural network chooses this action, very often the increase in its probability will be smaller than if the probability was low!
- # Note that if the action is chosen, very often it will also be update more often, and that could cause its probability to increase very quickly and make us stop exploring the rest of the actions.
- # To avoid this, we divide the gradient by its probability!

Since
$$\nabla \ln \pi(A_t | s_t, \theta_t) = \frac{\nabla \pi(A_t | s_t, \theta_t)}{\pi(A_t | s_t, \theta_t)}$$

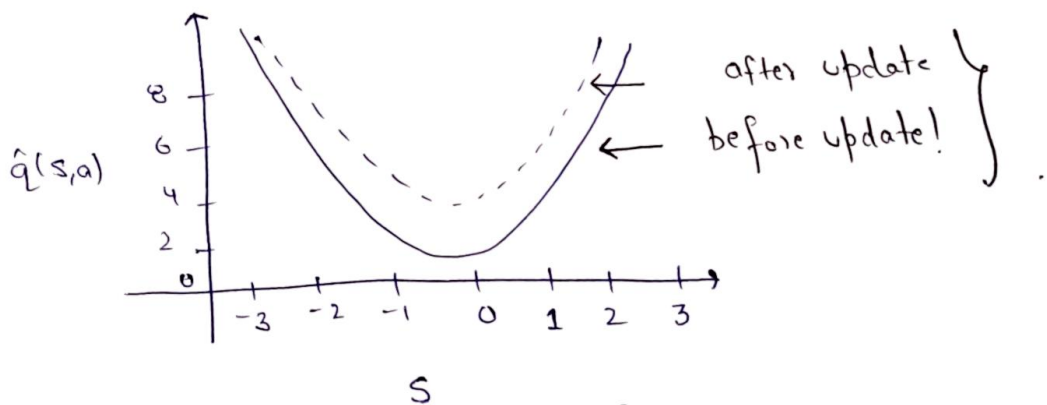
we have,

$$\underline{\theta_{t+1} = \theta_t + \alpha r^+ b_t \nabla \ln \pi(A_t | s_t, \theta_t)}$$

Parallel learning

(13)

- # When the agent performs the task, it takes actions that modify the state of the environment.
- # Normally, these actions have moderate effect on the state of the task.
- # What this means is that the successive states tend to be very similar to the previous ones, and this is the problem when we work with neural networks!
- # The problem is that when we update the value produced by a neural network, we are not only modifying the value but also nearby values!



- # If we update the neural network with a series of states that are very similar, we will be practically repeating the same update many times and thus the policy may ^{start} recommending some actions too often! — even in states where it's not good!

To fix this we have two options :-

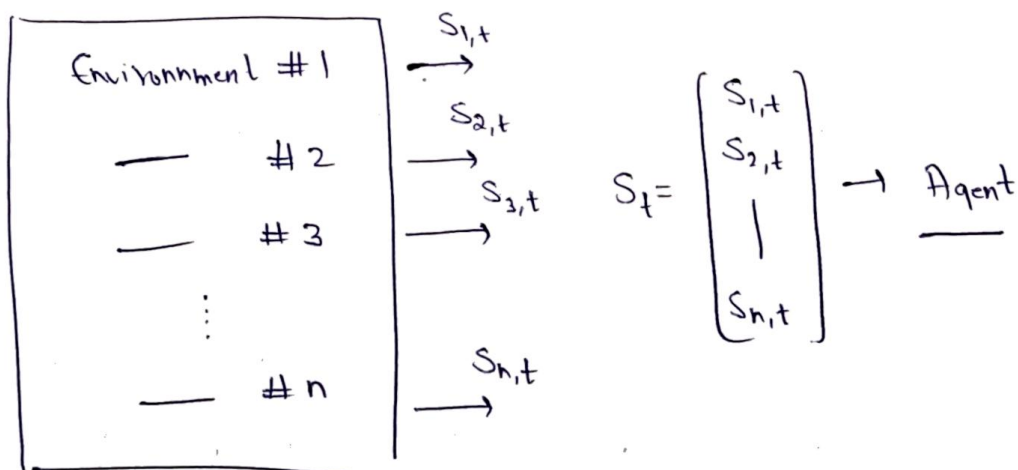
1) Replay memory :- When we use this technique, we update the neural network using batch of a randomly chosen experience. So when we

Note these experiences may belong to different episodes and completely different states!

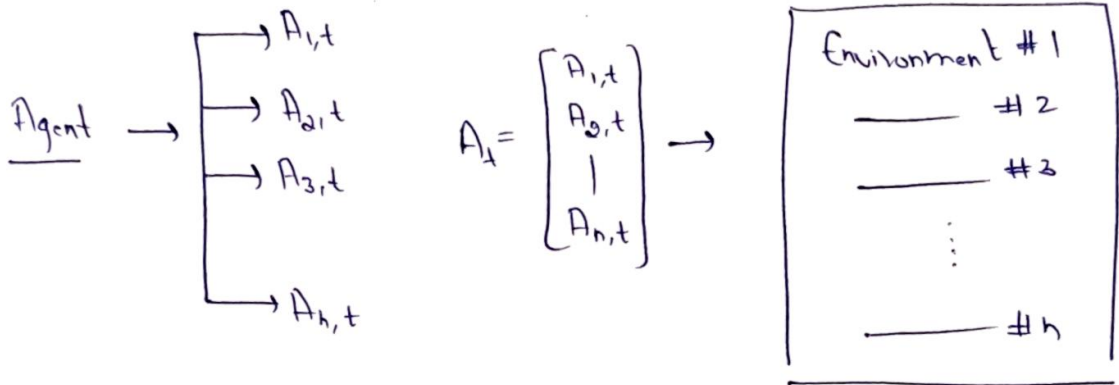
Update the neural network, we will modify it in a way that affects all these different states in a balanced way!

2) Parallel learning!

It consist of creating several environments in parallel with which the agent will interact at the same time!



- # When the environment starts the task, the agent will get one state for each one of the independent environments!



- # The agent will choose an action for each of the received states and will pass that action to the parallel environment!

Exploration Strategy!

- # We want to maintain the agent's exploration. We do not have mechanisms such as ϵ -greedy policies!
- # How to incorporate an exploration mechanism into our neural network!

Since in these methods,
neural network is
the policy!

which we
have for our value-
based algorithms

Solution —

- # We will incentivize the agent to keep the entropy of its policy as high as possible!

$$H(x) = - \sum_{x \in \mathcal{X}} p(x) \ln p(x)$$

It is the level of uncertainty of a random variable!
Variable!

- # In terms of our policy!

$$H_{\pi}(A_t) = - \sum_{a \in A_t} \pi(a|s_t) \cdot \ln(\pi(a|s_t))$$

Advantage Actor Critic (A2C) —

Policy Gradient + Temporal Difference

Advantage function :

Excess return of choosing the 'a' action instead of following the policy!

$$\boxed{Adv_n(s,a) = q_n(s,a) - v_n(s)}$$

- # Recall that the q-value of an action is the return that we get if take that action and then we follow the policy! On other hand, the value of a state is the return that we get

if we follow the policy!

If $\text{Adv}_{\pi}(s,a) > 0 \Rightarrow$ Taking action 'a' is better
than following the policy!

$\text{Adv}_{\pi}(s,a) < 0 \Rightarrow$ worse!

We want to ~~reinforce~~ reinforce the actions that obtain better results and discourage those that obtain worse results!

We can estimate 'advantage' as —

$$\boxed{\hat{\text{Adv}}_{\pi}(s,a) = \hat{q}(s,a) - \hat{v}(s)}$$

$$= r(s,a) + \gamma \hat{v}(s') - \hat{v}(s) \}$$

A2C

$$\Theta_{t+1} = \Theta_t + \alpha r^t \hat{\text{Adv}}_t \ln(\pi(A_t|s_t, \Theta_t)) \}$$

Reinforce

$$\Theta_{t+1} = \Theta_t + \alpha r^t G_t \ln(\pi(A_t|s_t, \Theta_t)) \}$$

A2C

$$\begin{array}{ccc} \underbrace{\text{Advantage}} & \underbrace{\text{Actor}} & \underbrace{\text{-Critic}} \\ \hat{A}dv(s,a) & \pi(a|s,\theta) & \hat{v}(s|\omega) \end{array}$$

we are going to use the advantage function to guide the optimization of the policy!

It will use a neural network that will act on the environment i.e. policy

we are going to use neural network that is going to estimate the values of the state and we are going to use it to refine the estimation of the advantage!

Update rule -

$$\theta_{t+1} = \theta_t + \alpha r^t \hat{A}dv_t \ln(\pi(A_t | s_t, \theta_t))$$

where, $\hat{A}dv_t = R_{t+1} + v \hat{v}(s_{t+1}|\omega) - \hat{v}(s_t|\omega)$ } Temporal error!